Python: pandas

Intro. to computing

`https://mandarm.github.io/computing.html`

Background: NumPy vs. Pandas

NumPy

- 'Clean', mostly numerical data
  (*structured arrays* also support general data)
- Elements indexed by non-negative integers

Pandas

- 'Noisy', real-world data with possibly missing entries
- Elements indexed by *labels* in addition to integers

Boilerplate

```
import numpy as np
import pandas as pd
```

Basic data structures: Series, DataFrames, Index

Series I

- Series $\equiv$ values + index
  - values / data: one-dimensional array holding items of the same type (integers / strings / Python objects)
  - *index*: sequence of labels (0, 1, ... by default)
- NumPy arrays $\approx$ lists; Pandas Series $\approx$ dicts (see examples below)
- May be created from lists, NumPy ndarrays, scalars, dicts, etc.
- Index may be explicitly specified.
  - scalars repeated as necessary
  - length has to match for lists, arrays
  - entries *do not* have to be distinct

```
# Default index (0, 1, ...)
data = pd.Series([0.25, 0.5, 0.75, 1.0])
print(f'Values: {data.values}, Index: {data.index}')
print(data[1:4], "\n", data[0])
```

Series II

```
# Explicit index with repeating elements
new_index = ("A", "B", "C", "D", "C")
new_data = pd.Series([0.25, 0.5, 0.75, 1.0, 1.25], index=new_index)
print(f'Values: {new_data.values}, Index: {new_data.index}')

# dict like behaviour
print("A" in new_data)
print(new_data["F"])
print(new_data.get("F"))

# non-dict-like behaviour
print(new_data['C'])      # 2 elements
print(new_data['B':'D'])  # slicing works
print(new_data['B':'C'])  # but not for repeated keys!

# From a scalar
x = pd.Series(5, index=[100, 200, 300])
```

Series III

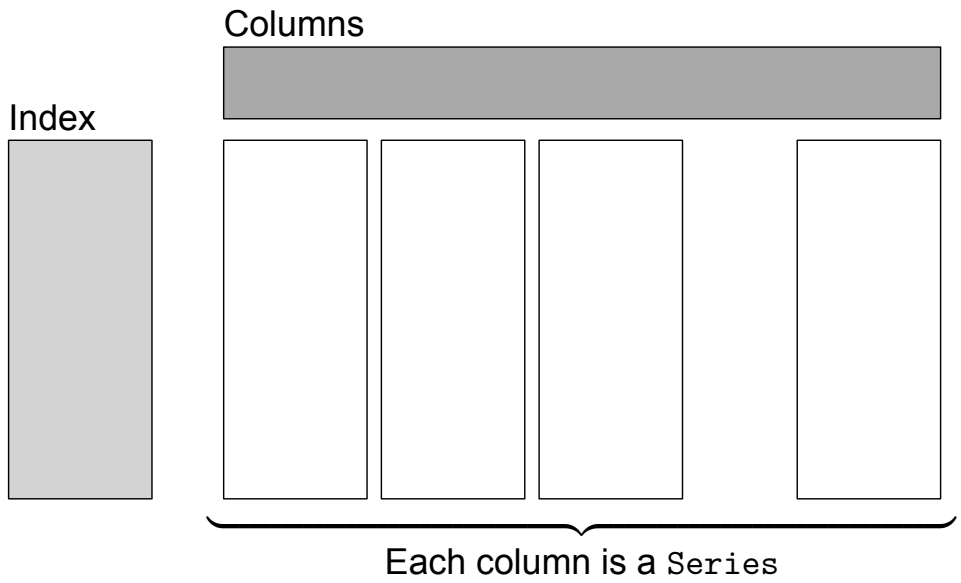
- Series from dictionaries
  - default index $\equiv$ dictionary keys
  - if index is explicitly provided, corresponding dict elements will be pulled out, and keys not in dict will be assigned NaN

```
# From a dictionary; default index  dict keys
y = pd.Series({2:'a', 1:'b', 3:'c'})
# may not be practical, but doable
y1 = pd.Series({2:'a', 1:'b', 3:'c', 'd':100})

# From a dictionary with explicit index  keys
z = pd.Series({2:'a', 1:'b', 3:'c'}, index=[3,2,10])
```

DataFrame I

- Two-dimensional array + row label (`index`) + column labels (`columns`)
  $\equiv$ sequence of `Series` objects that share the same index
- Different columns may have different types
- 2-D NumPy arrays $\approx$ lists of lists;
  Pandas DataFrames $\approx$ dicts (keys $\equiv$ column names, values $\equiv$ Series)
  of dicts (keys = index, values = values)



DataFrame II

- May be created from
  - 2-D ndarrays
  - dict with lists / dicts / ndarrays / Series as values

```
# From 2-D NumPy arrays
X = rng.random((4,3))
Y = pd.DataFrame(X)
print(X[0]), "\n", Y[0])

Z = pd.DataFrame(X, columns=['Bangalore', 'Delhi', 'Kolkata'],
                 index=list('abcd')])
```

Warning

- For 2-D `ndarrays`, `X[i]` is the i-th **row**.
- For `DataFrames`, `X[<label>]` is a **column**

DataFrame III

```
# From Series objects
population_dict = {'California': 39538223,
    'Texas': 29145505,
    'New York': 20201249,
    'Pennsylvania': 13002700,
    'Florida': 21538187, }
population = pd.Series(population_dict)

area_dict = {'California': 423967,
    'Florida': 170312,
    'New York': 141297,
    'Pennsylvania': 119280,
    'Texas': 695662, }
area = pd.Series(area_dict)

# Single-column DataFrame from single Series
dummy = pd.DataFrame(population, columns=['population'])
```

DataFrame IV

```
# Multi-column DataFrame from a dict of Series
states = pd.DataFrame({'population': population, 'area': area})
print(f'Row names: {states.index}\nColumn names: {states.columns}')
print(type(states['population'])) # Series

# Multi-column DataFrame from a dict of dicts
states = pd.DataFrame({'population': population_dict, 'area': area_dict
    })
print(f'Row names: {states.index}\nColumn names: {states.columns}')
print(type(states['population'])) # Series
```

DataFrame V

```
# Multi-column DataFrame from a heterogeneous dict
df2 = pd.DataFrame({
    "A": 1.0,     # scalars repeated as needed
    "C": pd.Series(1, index=list(range(4)), dtype="float32"), # these
    "D": np.array([3] * 4, dtype="int32"),                    # must be of
    "E": pd.Categorical(["test", "train", "test", "train"]),  # same length
    "F": "foo",   # # scalars repeated as needed
})
```

Warning

Creating DataFrame from **dict** of dicts and **list** of dicts is very different!

```
confusing_example1 = pd.DataFrame([population_dict, area_dict])
confusing_example2 = pd.DataFrame([{'a': 1, 'b': 2}, {'b': 3, 'c': 4}])
```

Dictionary keys become columns!

Index

- Index $\equiv$ *immutable* array or ordered (multi)set
- Standard indexing, slicing operations work.
- Share many attributes with NumPy arrays
  (`size`, `shape`, `dtype`)
- Supports set operations like union, intersection, difference

```
i1 = pd.Index([2, 3, 5, 7, 11])
i2 = pd.Index([1, 11, 9, 3, 2])

print(i1.union(i2))
print(i1.intersection(i2))
```

Reading and writing file data I

- Functions: `df = pd.read_X()`, `df.to_X()`
  where `X` may be `csv`, `excel`, `sql`, `json`, etc.
- `pd.read_csv(infile)`
  - `infile`: compulsory positional argument, may be
    - string, e.g. "/home/mandar/data/data.csv"
    - open file handle, e.g., `f = open(filename)`
  - `sep=` or `delimiter=` (default: comma (','))
  - `skiprows=`
    - number of lines to skip (single integer) at the start of the file, or
    - list of line numbers to skip (0-indexed)
  - `header=` line number that contains column names/labels (0-indexed)
    - counting starts after `skiprows` is processed
    - commented lines are also ignored (not counted)
    - pass `None` if column names is absent from input file
      (column names should be passed explicitly using `names`)

Reading and writing file data II

- `names=` user-defined column names
- `index_col=` label or index of column that should be used as row label
- and *many* more.

Attributes, operators, functions

DataFrames: attributes and basic functions

- `df.index`, `df.columns`, `df.T`
- `df.head(<optional no. of rows>)`: first few rows
  `df.tail(<optional no. of rows>)`: last few rows
- `df.describe()`: statistical summary
- `df.mean(axis=0)`, `df.var(axis=0)`

  Reminder: `axis=0` $\equiv$ column-wise operation, `axis=1` for row-wise operation
- `df.value_counts()`: Series containing frequency of each distinct row in `df`
  - `subset`: column label or list of labels (optional)
  - `normalize=False`: return relative frequencies instead of counts
  - `sort=True`: sort by frequencies (`False` $\Rightarrow$ sort by column values)
- `df.to_numpy()`: extract underlying NumPy array, without the index or column labels

Accessing elements I

Indexing (avoidable)

- `df["label"]`: returns a column as a Series
  - if label is valid identifier, `df.label` also works
- `df[["label2", "label3", "label1"]]`: returns multiple columns as a DataFrame

Accessing elements II

Using dedicated functions (recommended)

- `df.loc[<row selector>, <column selector>]`
  - column selector is optional
  - select rows and/or columns by single label / list of labels / label slice
  - to select all rows / columns, use `:` as selector
  - returns a *view*, not a copy
- `df.iloc[<row selector>, <column selector>]`
  - as above, but select rows and/or columns using single integer index / list of integer indices / slicing
- `df.at()`, `df.iat()`: similar to `loc/iloc`, but should be used when only single value needs to be obtained or set

Warning

For slicing with `loc[]`, **BOTH** start and stop are included.

Accessing elements III

Examples:

```
df = pd.DataFrame(rng.random((4,4)), columns=list("wxyz"))
print(df.loc[:, "x"])
print(df.loc[:, ["x", "z"]])    # In all these examples,
print(df.loc[:, "x":"z"])       # the column "z" is included
v = df.loc[2:3, "x":"z"]        # in the result.
v.loc[2,"y"] = 100

df.iloc[3]
df.iloc[3:5, 0:2]               # Usual slicing (stop not included)
df.iloc[[1, 2, 4], [0, 2]]
```

Boolean indexing: selection using conditions I

- For selecting elements from Series, Boolean expressions work as for ndarrays
- For selecting rows from DataFrame, Boolean vector must have same length as index

Example:

```
# With Series
s = pd.Series(range(-3, 4))
print(s[s > 0])
print(s[(s < -1) | (s > 0.5)])
print(s[~(s < 0)])
```

Boolean indexing: selection using conditions II

```
df = pd.DataFrame(rng.integers(50, size=(8, 4)), columns=list("ABCD")])
print(df[df['A'] > 0])

# More complex examples
df2 = pd.DataFrame({'a': ['one','one','two','three','two','one','six'],
                    'b': ['x', 'y', 'y', 'x', 'y', 'x', 'x'],
                    'c': np.random.randn(7)})

# Select only 'two' and 'three'
criterion = df2['a'].map(lambda x: x.startswith('t'))
print(df2[criterion])
# equivalent but slower
print(df2[[x.startswith('t') for x in df2['a']]])

# Multiple criteria
print(df2[criterion & (df2['b'] == 'x')])
print(df2.loc[criterion & (df2['b'] == 'x'), 'b':'c'])
```

Boolean indexing: selection using conditions III

Warning: `loc` vs. `iloc`

- For `loc`, selection can be done by array **or** Series.
- For `iloc`, selection can **only** be done by array, **not** Series.

  (use `.values` if needed)

```
df = pd.DataFrame([[1, 2], [3, 4], [5, 6]], index=list('abc'),
                  columns=['A', 'B'])
s = (df['A'] > 2)     # s is a Series
df.loc[s, 'B']        # works
df.iloc[s, 1]         # does not work
df.iloc[s.values, 1]  # works
```

Ufuncs I

- Similar to NumPy ufuncs
- For unary operations: index and column labels preserved
- For binary operations: *automatically aligns indices*
- Missing data $\Rightarrow$ NaN

Examples:

```
df = pd.DataFrame(rng.random((4,4)), columns=list("wxyz"))
print(np.sin(df * np.pi / 4))

# Examples with Series
area = pd.Series({'Alaska': 1723337, 'Texas': 695662, 'California':
    423967})
population = pd.Series({'California': 39538223, 'Texas': 29145505, '
    Florida': 21538187})
print(population / area) # Works on the union
# Compare area.index.union(population.index)
```

Ufuncs II

```
# Examples with DataFrames
A = pd.DataFrame(rng.integers(0, 20, (2, 2)), columns=['a', 'b'])
B = pd.DataFrame(rng.integers(0, 10, (3, 3)), columns=['b', 'a', 'c'])
print(A + B)   # handles "a" and "b" occurring in different orders
A.sum(B, fill_value=A.values.mean())
```

Python operator	Pandas method(s)
+	add
-	sub, subtract
*	mul, multiply
/	truediv, div, divide
//	floordiv
%	mod
**	pow

Names differ slightly from NumPy

Ufuncs III

```
A = pd.DataFrame(rng.integers(10, size=(3, 4)), columns=list("QRST"))
print(A - A.iloc[0])   # row-wise subtraction
print(A.subtract(A.iloc[:, "R"], axis=0)) # col-wise subtraction
halfrow = A.iloc[0, ::2]
print(A - halfrow)
# Try to explain: without axis=0, result is 3x7 array of NaNs
```

Making sense of the example:

- What is the type of `A.iloc[0]`, `halfrow`?
- What are their shapes?
- How would `A - A.iloc[0]` 'work' in NumPy?

Adding/deleting rows/columns I

- Assignment via `df.at`, `df.loc`

Examples:

```
df.loc[5,:] = rng.random(4)
df.loc[6,] = rng.random(4)
df.loc[3,"a"] = "hello"
```

  - missing values filled with NaN as usual
  - *will not work with* `df.iat`, `df.iloc` (index out of bound error)

Adding/deleting rows/columns II

- `pd.concat(<list of Series or DataFrames>, axis=0)`
  append rows or columns (with `axis=1`)

  Examples:

```
df1 = pd.DataFrame(rng.random((4,4)), columns=list("wxyz"))
df2 = pd.DataFrame(rng.random((4,4)), columns=list("wxyz"))
df3 = pd.DataFrame(rng.random((4,4)), columns=list("abcd"))
print(pd.concat([df1,df2]))           # Don't forget to put
print(pd.concat([df1,df3]))           # DFs to be concatenated
print(pd.concat([df1,df3],axis=1))    # in a list.
```

Note from the official documentation

Adding a column to a DataFrame is relatively fast. However, adding a row requires a copy, and may be expensive. We recommend passing a pre-built list of records to the DataFrame constructor instead of building a DataFrame by iteratively appending records to it.

Adding/deleting rows/columns III

- `df.drop(labels, *, axis=0, index=None, columns=None)`
  - `labels`: single row/column label or list of row/column labels (don't use tuples)
  - `index=L` may be used as replacement for `labels=L, axis=0`
  - `columns=C` may be used as replacement for `labels=C, axis=1`
- `df.pop(<column name>)`: return column as Series, and drop from frame

References

- *Python Data Science Handbook*, Jake VanderPlas, 2nd ed. (2023), O'Reilly Media; Part III, Chapters 13–16, 18, 19.
  https://jakevdp.github.io/PythonDataScienceHandbook/
- Official documentation
  https://pandas.pydata.org/docs/getting_started/intro_tutorials
- Cheatsheet:
  https://pandas.pydata.org/Pandas_Cheat_Sheet.pdf
- Topics to skip
  - special data types like date ranges
  - hierarchical / multi index
  - pivot tables
  - time series